Análisis y Diseño de Algoritmos

Grado de Ingeniería Informática

Universidad de Valladolid

Segunda Práctica: Compatibilidad de Contenedores en Cadenas de Suministro

Grupo: 31

Autoras:
Pérez Alonso, Marta
Rodrigues Álvarez, Emily Fabiana

Índice

1. Metodología **2**
- 1.1. Configuración para la realización de la práctica . . . 2
- 1.2. Entorno de desarrollo . . . 2

2. Estrategias de resolución por algoritmo **5**
- 2.1. Fuerza Bruta . . . 5
- 2.2. Voraz . . . 5
- 2.3. Programación Dinámica . . . 6

3. Resultados e interpretaciones **7**
- 3.1. Tablas de datos numéricos . . . 7
  - 3.1.1. Fuerza Bruta . . . 7
  - 3.1.2. Programación Dinámica . . . 7
  - 3.1.3. Voraz . . . 8
  - 3.1.4. Resumen de los datos observados . . . 8
- 3.2. Gráficas . . . 9
  - 3.2.1. Fuerza Bruta . . . 9
  - 3.2.2. Programación Dinámica . . . 10
  - 3.2.3. Voraz . . . 11
  - 3.2.4. Resumen de lás gráficas observadas . . . 11

4. Análisis de resultados **13**
- 4.1. Métodos de análisis . . . 13
- 4.2. Fórmulas obtenidas . . . 13
  - 4.2.1. Fuerza Bruta . . . 13
  - 4.2.2. Programación Dinámica . . . 14
  - 4.2.3. Voraz . . . 14
- 4.3. Impacto del Número de Atributos ($\boldsymbol{n}$) por Contenedor . . . 14
  - 4.3.1. Algoritmo Voraz y Programación Dinámica (PD) . . . 15
  - 4.3.2. Algoritmo de Fuerza Bruta . . . 15
  - 4.3.3. Resumen del impacto de n . . . 15

5. Conclusiones **16**
- 5.1. Análisis Comparativo de Complejidad . . . 16
- 5.2. Impacto del Número de Atributos . . . 16
- 5.3. Escalabilidad y Aplicabilidad . . . 16
- 5.4. Recomendación Final . . . 17

6. Porcentaje de trabajo individual **18**

7. Uso de inteligencia artificial generativa (IA-Gen) **19**
- 7.1. En el documento . . . 19
- 7.2. En el código . . . 19
- 7.3. En el cálculo de la complejidad . . . 19

8. Bibliografía y Fuentes **20**

1. Metodología

1.1. Configuración para la realización de la práctica

Para asegurar la validez y reproducibilidad de los resultados experimentales, se adoptó una configuración uniforme para todas las ejecuciones.

Antes de cada medición se invoca explícitamente el recolector de basura mediante `System.gc()`. Asimismo, todas las pruebas se ejecutan con el compilador JiT desactivado, evitando las variaciones introducidas por las optimizaciones dinámicas de la JVM en las primeras ejecuciones.

El análisis experimental se realizó para los tres algoritmos implementados (*Fuerza Bruta*, *Voraz* y *Programación Dinámica*). Para cada algoritmo y cada valor fijo del parámetro n (número de atributos del contenedor), se ejecuta el procedimiento 20 veces para cada valor de k (número de contenedores), calculando posteriormente la media aritmética del tiempo de ejecución. Este método reduce la variabilidad debida a factores externos y proporciona medidas temporales más estables y representativas.

Los resultados se almacenan en archivos CSV independientes. Cada archivo contiene:

- Un único algoritmo.
- Un valor fijo del parámetro n.
- Las medias temporales asociadas a los distintos valores de k.

Esta organización facilita el posterior análisis comparativo y la generación de gráficos de rendimiento.

Para el análisis del algoritmo de *Fuerza Bruta*, fue necesario restringir los valores de k al rango $[1, 10]$, ya que para valores superiores los tiempos de ejecución se disparaban. Aunque es posible ejecutar el algoritmo para $k = 11, 12, 13$, realizar 20 repeticiones para cada configuración suponía tiempos de ejecución de varias horas o incluso días, lo cual resultaba inviable para este estudio empírico.

Por el contrario, para los algoritmos *Voraz* y *Programación Dinámica* fue posible trabajar con valores de k en el rango $[10, 1000]$, lo que permitió observar mejor su crecimiento temporal. En particular, se realizaron mediciones para los valores: 10, 50, 100, 200, 300, ..., 1000.

En cuanto a los valores del parámetro n, para los algoritmos *Voraz* y *Programación Dinámica* se emplearon los siguientes: 3, 10, 30, 50 y 100. Esta elección se debe a que, en estos algoritmos, el factor que más influye en el orden asintótico es el número de contenedores k, mientras que el número de atributos n tiene un impacto menor.

Sin embargo, para el algoritmo de *Fuerza Bruta* fue necesario limitar los valores de n a 3, 10 y 30, debido al crecimiento rápidamente escalado asociado al aumento del número de contenedores, que hace que el tamaño de las combinaciones crezca rápidamente y afecte de forma significativa al tiempo de ejecución.

1.2. Entorno de desarrollo

El desarrollo de la práctica se ha realizado en `Java`. El proyecto mantiene una estructura modular que separa el código fuente, los archivos compilados, los datos de entrada y los resultados experimentales.

Por un lado, en la carpeta `src` se encuentran todos los archivos `.java` que conforman el código ejecutable. Entre ellos se incluyen las clases `Main`, `FuerzaBruta`, `ProgramacionDinamica` y `Voraz`, que contienen la lógica principal del análisis. Para

unificar las funciones necesarias de cada algoritmo, se definió la interfaz `IMedicion`, que debe ser implementada por cada una de las clases correspondientes.

Asimismo, la clase `Contenedor` almacena los atributos de cada contenedor y proporciona la función `esCompatibleCon`, que ordena los atributos de ambos contenedores en orden creciente y comprueba si todos los atributos del primero son estrictamente menores que los del segundo.

El directorio `inputs` contiene todos los archivos de entrada generados. Se divide en dos subdirectorios: `analisis` y `pruebas`.

En `analisis`, tal y como indica su nombre, se encuentran los ficheros utilizados para el estudio experimental. Este directorio incluye una subcarpeta `fb` (para el algoritmo de *Fuerza Bruta*), subdividida a su vez en carpetas correspondientes a los distintos valores de n empleados; dentro de cada una de ellas se ubican los ficheros con diferentes valores de k. De manera equivalente, existen subdirectorios para los otros algoritmos, organizados también por valores de n y con ficheros generados para múltiples valores de k.

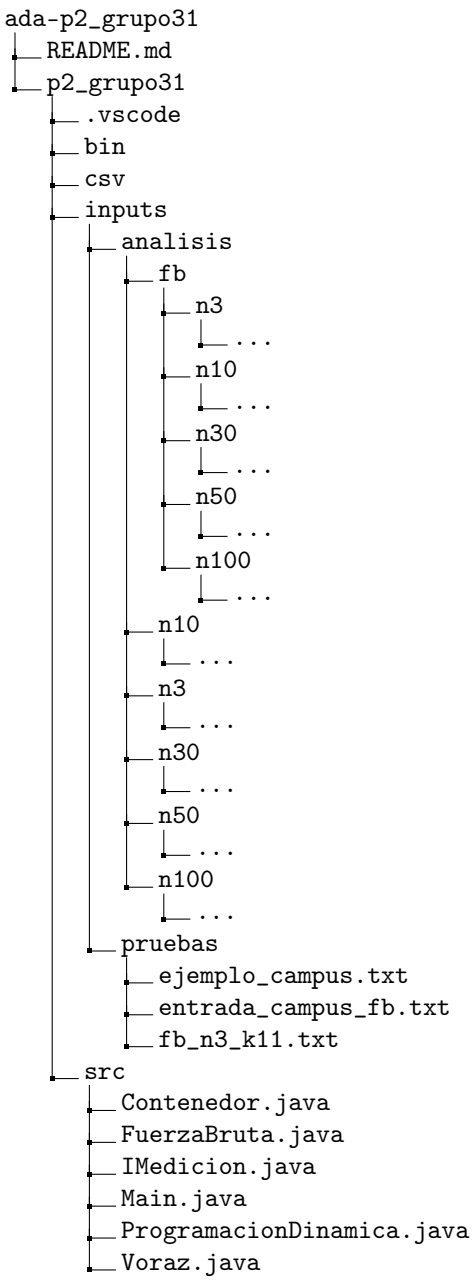
Para compilar y ejecutar el proyecto se siguen las instrucciones incluidas en el archivo `README.md`. De forma resumida, el procedimiento consiste en:

1. Cambiar al directorio `p2_grupo31`.
2. Compilar los archivos `.java` del directorio `src`, generando los `.class` en `bin`.
3. Acceder al directorio `bin`.
4. Ejecutar la clase `Main` con la compilación JiT desactivada para evitar optimizaciones que puedan afectar a las mediciones de tiempo.

Las mediciones se realizan mediante las clases `Instant` y `Duration`, que permiten obtener el tiempo en nanosegundos utilizando exclusivamente bibliotecas estándar de Java.

Si bien para el análisis de `Fuerza Bruta` cuenta con distintos valores de n, tuvimos que cancelar la ejecución para los fichero con $n \geq 50$ porque pasaba muchas horas ejecutando hasta $3 \leq n \leq 30$ y no pudimos modificar el `Main` para esta nueva configuración.

La estructura completa del proyecto es la siguiente:


ada-p2_grupo31
README.md
p2_grupo31
.vscode
bin
csv
inputs
analisis
fb
n3
...
n10
...
n30
...
n50
...
n100
...
n10
...
n3
...
n30
...
n50
...
n100
...
pruebas
ejemplo_campus.txt
entrada_campus_fb.txt
fb_n3_k11.txt
src
Contenedor.java
FuerzaBruta.java
IMedicion.java
Main.java
ProgramacionDinamica.java
Voraz.java

2. Estrategias de resolución por algoritmo

2.1. Fuerza Bruta

El algoritmo de *Fuerza Bruta* explora todas las permutaciones posibles de los contenedores para encontrar la secuencia más larga de contenedores compatibles. Su implementación se basa en los siguientes principios:

1. **Generación de permutaciones**: Se generan todas las permutaciones posibles del conjunto de contenedores mediante un algoritmo recursivo de retroceso (*backtracking*). Para ello, se utiliza un array `contenedoresUsados` que registra qué contenedores ya han sido incluidos en la permutación actual.

2. **Evaluación de cada permutación**: Para cada permutación completa generada, se evalúa la longitud de la secuencia más larga de contenedores compatibles consecutivos. Esto se realiza recorriendo la permutación y verificando la compatibilidad entre cada par de contenedores adyacentes mediante el método `esCompatibleCon`.

3. **Actualización de la mejor solución**: Durante la evaluación, se mantiene un registro de la mejor solución encontrada hasta el momento. Si una permutación produce una secuencia más larga que la mejor conocida, se actualiza tanto la longitud como los índices de los contenedores que forman dicha secuencia.

2.2. Voraz

El algoritmo *Voraz* adopta una estrategia heurística que construye una solución de forma incremental, tomando en cada paso la decisión localmente óptima sin reconsiderar elecciones previas. Su funcionamiento se estructura en las siguientes fases:

1. **Criterio de ordenación**: Los contenedores se ordenan según la suma de sus atributos, de menor a mayor. Esta heurística se fundamenta en la intuición de que comenzar con los contenedores "más pequeños" facilita la construcción de cadenas más largas, ya que existe mayor probabilidad de encontrar contenedores compatibles posteriormente. Se había intentado ordenar de manera ascedente los contenedores a partir de su atributo de menor valor, pero obteníamos resultados de poco valor a nivel de solución. Planteamos realizar el ordenamiento según la media geométrica de cada contenedor, igualmente de forma ascedente, y en caso de empate eligiese aquel contenedor con menor varianza. Sin embargo, no utilizamos este crititerio porque significaba implementar varios arrays o array de arrays.

2. **Construcción voraz de la secuencia**: Una vez ordenados, se inicia la secuencia con el contenedor de menor suma. A continuación, se recorre el array ordenado y se añade cada contenedor que sea compatible con el último contenedor incluido en la secuencia.

3. **Algoritmo de ordenación**: La ordenación se implementa mediante *Ordenación Rápida* (*QuickSort*), seleccionando un pivote aleatorio para evitar el peor caso. La comparación entre contenedores se basa en la suma de sus atributos, calculada mediante el método auxiliar `suma`.

2.3. Programación Dinámica

El algoritmo de *Programación Dinámica* resuelve el problema de forma óptima mediante la construcción de soluciones parciales que se combinan para obtener la solución global. Se estructura de la siguiente manera:

1. **Ordenación previa**: Al igual que en el algoritmo *Voraz*, los contenedores se ordenan según la suma de sus atributos en orden creciente. Esta ordenación es esencial para garantizar que, si un contenedor C_j es compatible con C_i, entonces C_j aparecerá antes que C_i en el array ordenado, permitiendo aplicar correctamente la programación dinámica.

2. **Definición de la tabla de programación dinámica**: Se define un array `L[i]` que almacena la longitud de la secuencia más larga de contenedores compatibles que termina en el contenedor i. Adicionalmente, se mantiene un array `Previo[i]` que registra el índice del contenedor anterior en dicha secuencia óptima.

3. **Relación de recurrencia**: Para cada contenedor i, se evalúan todos los contenedores $j < i$ y se verifica si C_j es compatible con C_i. Si lo es, se actualiza `L[i]` según la relación:

   $$L[i] = \text{máx}(L[i], L[j] + 1)$$

   Esta ecuación refleja que la mejor secuencia que termina en i puede extenderse desde la mejor secuencia que termina en j.

4. **Recuperación de la solución**: Una vez completada la tabla `L`, se identifica el índice `maxLenIdx` donde se alcanza la longitud máxima. A partir de este índice, se reconstruye la secuencia óptima utilizando el array `Previo`, recorriendo la cadena de predecesores desde el final hasta el inicio.

3. Resultados e interpretaciones

3.1. Tablas de datos numéricos

3.1.1. Fuerza Bruta

k	n=3	n=10	n=30
1	39837	41775	32362
2	62887	57400	82937
3	85412	99062	221125
4	19465	82905	599860
5	563795	1267835	5035800
6	3363025	8684835	36816140
7	25606725	72165545	325961500
8	218542110	552346830	3008255920
9	2012627775	5065805055	29745182860
10	20781004350	59833006170	325036696700

Tabla 1: Tiempo de ejecución ($nseg$) — Fuerza Bruta

El crecimiento temporal es explosivo, aumentando aproximadamente 10 con cada incremento unitario de

Para $k = 10$ y $n = 30$, el tiempo supera los 5 minutos (325 mil millones de nanosegundos).

El parámetro n multiplica linealmente el tiempo: con $n = 30$, los tiempos son aproximadamente $5 - 10$ mayores que con $n = 3$.

Esto explica y demuestra por qué el análisis para `Fuerza Bruta` tiene un rango de valores para probar diferente al resto de algoritmos de la práctica.

3.1.2. Programación Dinámica

k	n=3	n=10	n=30	n=50	n=100
10	762083	674891	1624812	2549566	5118962
50	4410100	8824458	31045500	39584066	68428604
100	13899595	29336975	82548625	95882362	189092262
200	35657970	58335566	237449270	304376225	704840095
300	58146470	128566766	517544100	674356158	1565940212
400	100297187	186446175	901149954	1199394141	2739168495
500	131476441	273558695	1433857387	1853439191	4313450879
600	177739558	386986562	1944000312	2528963741	6398934358
700	215738945	513854450	2655014883	3575434141	8405576100
800	252085620	654734733	3501180200	4568966770	11017652237
900	316484362	858972737	4408276745	5785708975	13617440054
1000	375429508	995953504	5371235045	7108862650	17509366604

Tabla 2: Tiempo de ejecución ($nseg$) — Programación Dinámica

Se observa un crecimiento cuadrático respecto a k: al duplicar k (de 100 a 200, de 500 a 1000), el tiempo se multiplica aproximadamente por 4.

El efecto de n es lineal: duplicar n duplica aproximadamente el tiempo de ejecución (comparar columnas $n = 50$ y $n = 100$).

Para $k = 1000$ y $n = 100$, el tiempo es de 17.5 segundos, lo que demuestra que el algoritmo es viable para problemas de tamaño real.

3.1.3. Voraz

k	**n=3**	**n=10**	**n=30**	**n=50**	**n=100**
10	197425	263133	562966	686604	1209008
50	503470	747491	2121458	2388425	4776112
100	783858	1406533	4084604	5404583	11017691
200	1498470	2581858	7540416	10333379	20999583
300	1998554	3146254	11269741	16564579	32035745
400	2616225	4833704	14246066	21337141	36582800
500	3189025	5845208	17983508	26129212	45671366
600	3770916	6667083	23282391	30132379	54456750
700	4310204	8398145	28394454	33797275	54966045
800	4913162	9775475	29973754	38371304	63344716
900	5230233	10852479	31584333	43814216	58976691
1000	6161358	12054625	36586008	45622012	70637945

Tabla 3: Tiempo de ejecución ($nseg$) — Voraz

El algoritmo *Voraz* muestra tiempos significativamente menores que *Programación Dinámica*: para $k = 1000$, $n = 100$, el tiempo es 70 millones de nanosegundos (0.07 seg) frente a 17.5 segundos de *Programación Dinámica*.

El crecimiento también parece cuadrático en k, aunque con una constante multiplicativa menor.

El comportamiento respecto a n es lineal, similar a *Programación Dinámica*.

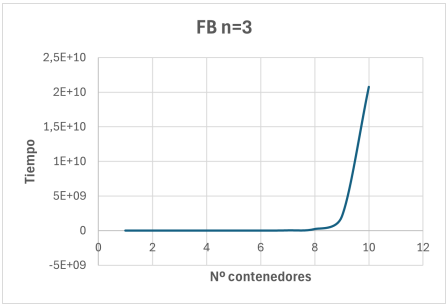
La eficiencia superior se debe a que *Voraz* no explora todas las combinaciones, sino que toma decisiones locales.

3.1.4. Resumen de los datos observados

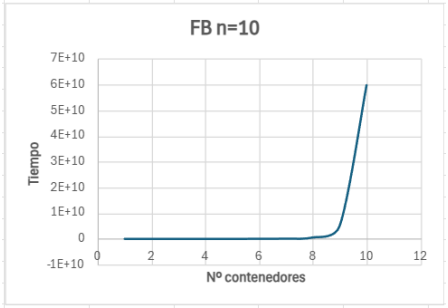
Los datos numéricos recopilados en las tablas 1 a 3 confirman las diferencias de complejidad asintótica teóricas entre los tres algoritmos, las cuales están influenciadas principalmente por el número de contenedores (k) y, secundariamente, por el número de atributos (n).

3.2. Gráficas

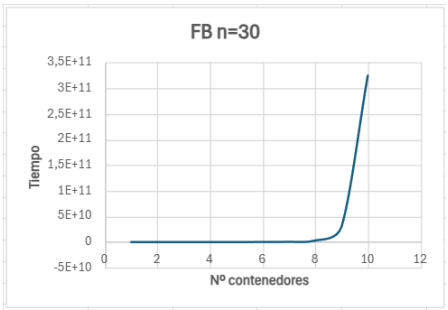
3.2.1. Fuerza Bruta



(a) Nº Atributos = 3



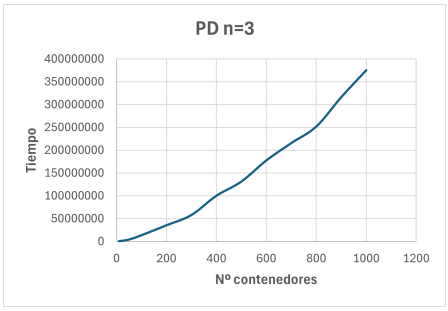
(b) Nº Atributos = 10



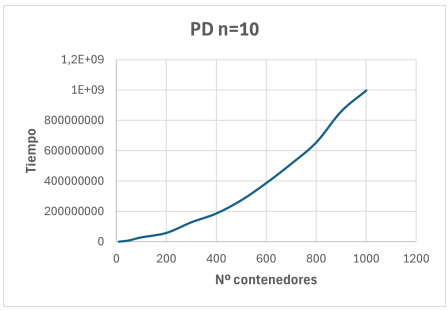
(c) Nº Atributos = 30

Figura 1: Gráficas del algoritmo de Fuerza Bruta (tiempo promedio en nanosegundos)

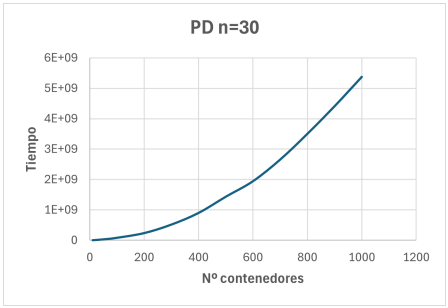
3.2.2. Programación Dinámica



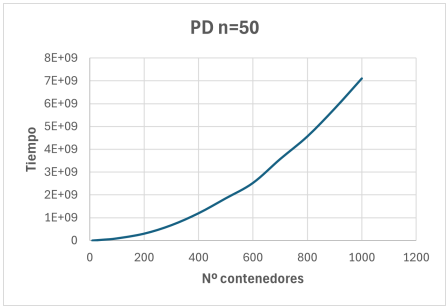
(a) Nº Atributos = 3



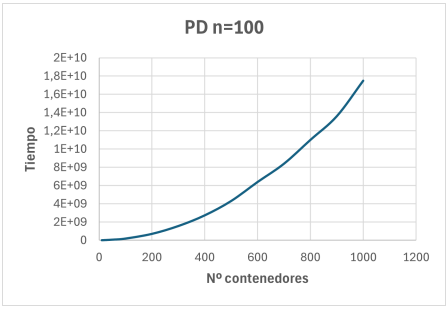
(b) Nº Atributos = 10



(c) Nº Atributos = 30



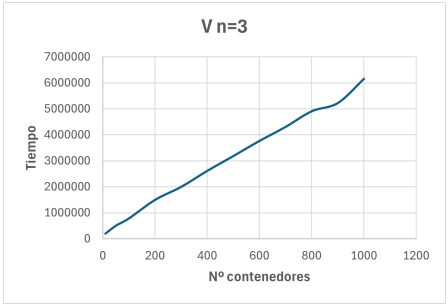
(d) Nº Atributos = 50



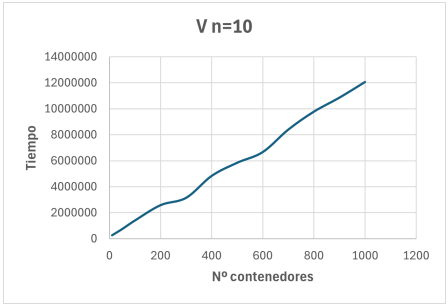
(e) Nº Atributos = 100

Figura 2: Gráficas del algoritmo de Programación Dinámica (tiempo promedio en nanosegundos)

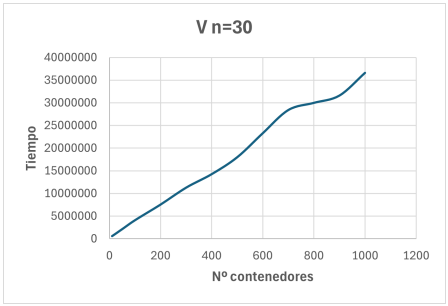
3.2.3. Voraz



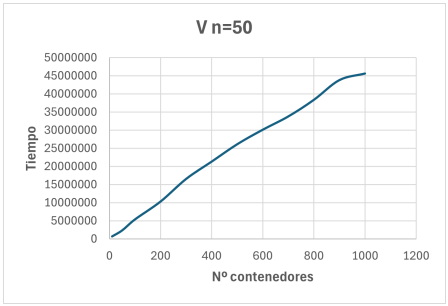
(a) Nº Atributos = 3



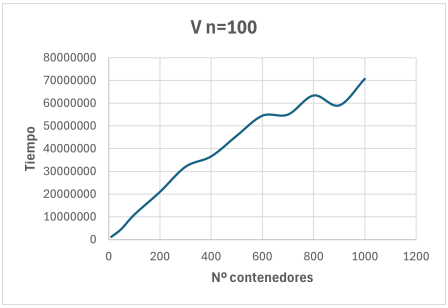
(b) Nº Atributos = 10



(c) Nº Atributos = 30



(d) Nº Atributos = 50



(e) Nº Atributos = 100

Figura 3: Gráficas del algoritmo Voraz (tiempo promedio en nanosegundos)

3.2.4. Resumen de lás gráficas observadas

Las gráficas presentadas confirman visualmente los patrones identificados en las tablas.

Para *Fuerza Bruta*, el patrón es de crecimiento altamente avanzado es evidente en todas las configuraciones de n. El tiempo se dispara bruscamente a partir de un número muy bajo de contenedores (aproximadamente 10), lo que lo convierte en un algoritmo inviable para cualquier caso de tamaño medio o grande.

Por otro lado, *Programación Dinámica* presenta un comportamiento mucho más regular. Este patrón de crecimiento controlado le permite manejar rangos de hasta 1000 contenedores, aunque el tiempo de ejecución es significativamente mayor que el de *Voraz*.

Asimismo, *Voraz* tiene consistencia con un crecimiento también regular, incluso más que en *Programación Dinámica* en el tiempo de ejecución. Este patrón se man-

tiene a lo largo de las diferentes métricas y configuraciones de n, lo que lo posiciona como el algoritmo más eficiente y escalable.

4. Análisis de resultados

4.1. Métodos de análisis

Para determinar las fórmulas que mejor se ajustan a los datos experimentales, seguimos el siguiente procedimiento considerando distintos tipos de funciones candidatas seg'un el algoritmo:

- **Fuerza Bruta**: $k!$.
- **Voraz**: k.
- **Programación Dinámica**: $k\ln(k)$.

Al estar utilizando excel, tuvimos que:

1. Crear columnas con los valores de las funciones candidatas para cada tamaño k.
2. Dividir los datos obtenidos (tiempo de ejecución promedio) entre cada una de estas funciones.
3. Si al dividir los datos entre una función el resultado se mantiene aproximadamente constante, probablemente esa función sea la que describe el comportamiento del algoritmo.
4. Para verificar, calculamos el coeficiente de variación (CV) para cada función candidata:
   - $CV = \frac{\sigma}{\bar{x}} 100\,\%$
   - Una variación pequeña ($< 10\,\%$) indica que la función describe correctamente el crecimiento de los datos.
5. Para calcular la constante que acompaña a la función, calculamos el promedio de los cocientes.

4.2. Fórmulas obtenidas

Los cálculos de esta sección se han obtenido directamente de las medias de tiempo presentadas en las Tablas 1–3 (valores en nanosegundos) manteniendo exactamente los mismos valores de k y n que aparecen en dichas tablas. Para cada posible función candidata $f(k)$ (p. ej. $k!$, $kln(k)$, ...) se han calculado los cocientes $T(k,n)/f(k)$, la variación de los cocientes para ver cuál de las funciones se adaptaba más y su media. Los resultados relevantes se muestran a continuación (unidades: ns/$f(k)$). Los datos originales están en las Tablas del informe.

4.2.1. Fuerza Bruta

Se ha utilizado el conjunto $k = 1, \ldots, 10$ tal y como aparece en la Tabla 1. Para cada n se calculó $C(k) = T(k,n)/k!$ y sus estadísticos:

Observaciones: la variabilidad relativa es alta para n pequeños (CV $> 100\,\%$ para $n = 3$), lo cual es esperable dado el rápido crecimiento y la sensibilidad a pequeñas variaciones en tiempos cuando se divide por $k!$.

n	$\bar{C}$ (ns / $k!$)	σ (ns / $k!$)
3	$1{,}1747 \times 10^4$	$112,13$
10	$1{,}7153 \times 10^4$	$62,29$
30	$5{,}3960 \times 10^4$	$41,43$

Tabla 4: Estadísticos de $T/k!$ para Fuerza Bruta (valores extraídos de Tabla 1).

4.2.2. Programación Dinámica

Se ha usado el conjunto $k = \{10, 50, 100, 200, 300, 400, 500, 600, 700, 800, 900, 1000\}$ (tal y como en la Tabla 2). Para cada valor de n se calculó $C(k) = T(k, n)/kln(k)$ y se obtuvieron los estadísticos:

n	$\bar{C}$ (ns / $kln(k)$)	σ (ns / $kln(k)$)
3	$40346,37$	$23,81$
10	$87828,98$	$42,11$
30	$417527,00$	$56,39$
50	$547225,06$	$57,03$
100	$1284469,14$	$61,26$

Tabla 5: Estadísticos de $T/kln(k)$ para Programación Dinámica (valores extraídos de Tabla 2).

4.2.3. Voraz

Se ha utilizado el mismo conjunto de k que para Programación Dinámica (Tabla 3). Los estadísticos de T/k son:

n	$\bar{C}$ (ns / k)	σ (ns / k)
3	$7939,98$	$49,06$
10	$13495,11$	$31,20$
30	$39578,02$	$14,46$
50	$51977,43$	$11,55$
100	$92151,34$	$18,13$

Tabla 6: Estadísticos de T/k para Voraz (valores extraídos de Tabla 3).

4.3. Impacto del Número de Atributos (n) por Contenedor

El número de atributos por contenedor (n) es un factor que influye directamente en el tiempo de ejecución de un algoritmo debido al costo de la operación de verificación de compatibilidad, la cual tiene una complejidad mínima de $O(n)$.

A continuación, se resume el impacto de n en los tres algoritmos, basándose en los datos experimentales para valores fijos del número de contenedores (k):

4.3.1. Algoritmo Voraz y Programación Dinámica (PD)

En ambos algoritmos, el aumento en el número de atributos (n) conlleva un aumento significativo del tiempo de ejecución. Esto se debe a que la complejidad temporal de estos algoritmos contiene un factor dependiente de n que no es despreciable.

Tabla 7: Impacto de n en Voraz y PD (con $k = 100$ contenedores fijos)

Algoritmo	n	**Tiempo Promedio (ns)**	**Factor de Aumento ($n = 100/n = 3$)**
Voraz	3	783,858	$\approx 14,05$
	100	11,017,691	
PD	3	13,899,595	$\approx 13,60$
	100	189,092,262	

- **Crecimiento** El aumento de n de 3 a 100 (un factor de $\approx 33,3$) provoca un incremento en el tiempo de ejecución de ≈ 14 veces, lo que indica una fuerte dependencia de n en la función de complejidad del tiempo.
- **Causa:** Se relaciona con términos de complejidad como que hacen que el algoritmo no sea totalmente insensible a la dimensión de los contenedores.

4.3.2. Algoritmo de Fuerza Bruta

El impacto de n en el algoritmo de Fuerza Bruta es mucho menor en comparación con el efecto que tiene la cantidad de contenedores (k), cuya complejidad es factorial ($O(k!)$).

Tabla 8: Impacto de n en Fuerza Bruta (con $k = 3$ contenedores fijos)

Algoritmo	n	**Tiempo Promedio (ns)**	**Factor de Aumento ($n = 30/n = 3$)**
Fuerza Bruta	3	85,412	$\approx 2,59$
	30	221,125	

- **Crecimiento:** El tiempo de ejecución se incrementa, pero de manera relativamente modesta ($\approx 2,6$ veces) al aumentar n en un factor de 10.
- **Causa:** La complejidad del algoritmo está dominada por el término $O(k!)$ (permutaciones de contenedores), lo que minimiza el impacto del costo $O(n)$ de la verificación de compatibilidad dentro de cada permutación. Por lo tanto, el algoritmo es más sensible a k que a n.

4.3.3. Resumen del impacto de n

En general, el tamaño evidentemente afecta a los algoritmos pero no en mayor medida que el número de contendores, en especial para `Fuerza Bruta` donde el número de contenedores domina casi por completo el tiempo de ejecución.

5. Conclusiones

En esta práctica se han implementado y analizado tres algoritmos diferentes para resolver el problema de compatibilidad de contenedores en cadenas de suministro: Fuerza Bruta, Voraz y Programación Dinámica. A continuación se presenta un análisis comparativo de los resultados obtenidos.

5.1. Análisis Comparativo de Complejidad

Fuerza Bruta muestra un comportamiento proporcional a $k!$, como se deduce del análisis empírico realizado mediante el cociente $T(k, n)/k!$. Los resultados experimentales confirman este comportamiento explosivo: para $k = 10$ contenedores con $n = 30$ atributos, el tiempo de ejecución supera los 325 segundos. El algoritmo explora todas las permutaciones posibles, garantizando la solución óptima pero resultando inviable para casos reales incluso con tamaños moderados. El factor n tiene impacto lineal, multiplicando los tiempos por aproximadamente 2.6 al pasar de $n = 3$ a $n = 30$.

Programación Dinámica presenta un comportamiento proporcional a $k \cdot \ln(k)$, según el análisis del cociente $T(k, n)/[k \cdot \ln(k)]$, que muestra coeficientes de variación relativamente bajos (23–61 %). El comportamiento cuadrático respecto a k es evidente en los datos experimentales: al duplicar k de 500 a 1000, el tiempo se multiplica aproximadamente por 4. Para $k = 1000$ con $n = 100$, el algoritmo requiere 17.5 segundos, lo que demuestra su viabilidad para problemas de tamaño real. Este algoritmo garantiza encontrar la solución óptima gracias al principio de optimalidad, construyendo la solución mediante la combinación de subsecuencias óptimas.

Voraz muestra comportamiento lineal respecto a k, como indica el análisis del cociente $T(k, n)/k$, con coeficientes de variación bajos (11–49 %). Los tiempos son significativamente menores que Programación Dinámica: para $k = 1000$ con $n = 100$, el tiempo es de 0.07 segundos frente a los 17.5 segundos de Programación Dinámica (aproximadamente 250 veces más rápido). Sin embargo, este algoritmo no garantiza encontrar la solución óptima global, ya que toma decisiones locales basándose en la heurística de ordenar por suma de atributos.

5.2. Impacto del Número de Atributos

El número de atributos (n) influye de manera diferente en cada algoritmo:

- En **Voraz** y **Programación Dinámica**, el efecto es aproximadamente lineal. Al aumentar n de 3 a 100, el tiempo se incrementa en un factor de ~ 14, debido principalmente al coste $O(n)$ de verificar compatibilidad entre contenedores.
- En **Fuerza Bruta**, aunque n también afecta al tiempo de ejecución (factor $\sim 2{,}6$ al pasar de $n = 3$ a $n = 30$), su impacto es mucho menor comparado con el efecto dominante de $k!$, que crece de forma factorial.

5.3. Escalabilidad y Aplicabilidad

Fuerza Bruta solo es aplicable a problemas de tamaño muy pequeño ($k \leq 10$). Su uso se justifica únicamente cuando se requiere garantizar la optimalidad absoluta en casos triviales o cuando la simplicidad de implementación es prioritaria sobre la eficiencia.

Programación Dinámica es adecuado para problemas de tamaño medio a grande donde se requiere la solución óptima. Maneja eficientemente hasta $k = 1000$ conte-

nedores con tiempos razonables. Es la mejor opción cuando la optimalidad es requisito imprescindible y el tamaño del problema no es excesivamente grande.

Voraz es el algoritmo más escalable y eficiente, ideal para problemas de gran tamaño donde los tiempos de respuesta son críticos. Aunque no garantiza optimalidad, su rendimiento superior lo convierte en la opción práctica para sistemas en tiempo real o con grandes volúmenes de datos.

5.4. Recomendación Final

Para el problema de compatibilidad de contenedores, la elección del algoritmo depende del contexto:

- Si el tamaño del problema es muy pequeño ($k < 10$) y se requiere garantía absoluta de optimalidad, **Fuerza Bruta** es suficiente.
- Para problemas de tamaño moderado ($k \leq 1000$) donde la solución óptima es esencial, **Programación Dinámica** es la mejor elección, ofreciendo un equilibrio razonable entre optimalidad y eficiencia.
- Para aplicaciones industriales con grandes volúmenes de datos o requisitos de tiempo real, **Voraz** es el algoritmo recomendado, proporcionando soluciones de alta calidad en tiempos prácticamente despreciables.

En términos generales, los resultados experimentales validan las predicciones teóricas de complejidad y demuestran la importancia de seleccionar la técnica algorítmica apropiada según las restricciones específicas de cada problema.

6. Porcentaje de trabajo individual

Consideramos que hemos distribuido el trabajo equitativamente entre ambas, con una participación del 50 % cada una.

Emily Fabiana Rodrigues Álvarez 50 %

Marta Pérez Alonso 50 %

7. Uso de inteligencia artificial generativa (IA-Gen)

Si bien se ha hecho uso de la inteligencia artificial generativa (IA-Gen), cada consulta y respuesta generadas fueron evaluadas antes de incluirlas en este análisis.

7.1. En el documento

En el desarrollo de este documento, se ha utilizado inteligencia artificial generativa (IA-Gen) como herramienta de apoyo para:

- Estructurar el documento mediante consultas para organizar las secciones de manera académica y coherente.
- Ayudarnos con el formato de LaTeX para la construcción de las tablas, así como para la adición de las gráficas.
- Hacer correcciones ortográficas y mejorar la comprensión del texto escrito.

En concreto, se consultó a `DeepSeek`, `ChatGPT` y `Gemini`.

7.2. En el código

Para la generación de los ficheros de entrada, especificando la cantidad de atributos de los contenedores y el número de contenedores, se utilizó `GitHub Copilot`.

También se hizo uso de la IA-Gen para la creación de los archivos `CSV` y el formateo de almacenamiento de datos en el mismo. Así como para la lectura de los ficheros de entrada. En concreto, se consultó a `Claude`, `ChatGPT` y `Gemini`.

Asimismo, se utilizó `GitHub Copilot` para la documentación del código.

7.3. En el cálculo de la complejidad

Se utilizó inteligencia artificial como apoyo durante el análisis en Excel, principalmente para obtener la fórmula del coeficiente de variación y aplicarla correctamente a los datos.

En concreto, se consultó a `Claude` y `ChatGPT`.

8. Bibliografía y Fuentes

Material de la asignatura

- Tema 2 - Diseno de Algoritmos. Departamento de Informática, Universidad de Valladolid. González Ferreras, César.

  Utilizado para: la selección de los tres algoritmos usados (*Fuerza Bruta*, *Voraz* y *Programación Dinámica*), así como para saber el orden de cada algoritmo y verificar si lo cumplen.